

Pointers and Objects

*Data Structures and Algorithm Analysis in C, by Mark Allen Weiss, 2nd
edition, 1997, Addison-Wesley, ISBN 0-201-49840-5*
Danang University of Science and Technology

Dang Thien Binh
dtbinh@dut.udn.vn

Readings and References

■ Reading

■ Other References

- ▶ *Pointers and Memory*, by Parlante
- ▶ Chapters 5 and 6, *The C Programming Language*, Kernighan and Ritchie

What is a pointer?

- A pointer is a reference to another item
 - ▶ the other item is sometimes called the "pointee"
 - ▶ the pointee is often a variable
 - ▶ the pointee can also be a function (a procedure)
- The contents of a pointer tell you where to look in order to find the object of interest
- The declaration of the pointer says what it is supposed to point to

Some declarations

```
int num;           /* an integer */
int *numP;         /* pointer to integer */

double sum;        /* a double value */
double *sumP;      /* pointer to double */

struct Symbol mySym; /* a Symbol */
struct Symbol *symP  /* pointer to Symbol */
```

Reference and dereference

- Use "&" operator to take the address of a variable and store it in a pointer variable
 - ▶ `numP = #`
 - ▶ `sumP = ∑`
 - ▶ `symP = &mySym;`
- Then use "*" operator to dereference a pointer
 - ▶ `*numP = 42` is the same as `num = 42`
 - ▶ `*sumP = 17.0` is the same as `sum = 17.0`
 - ▶ `(*symP).val = 2` is the same as `mySym.val = 2`

Pointers and Pointees

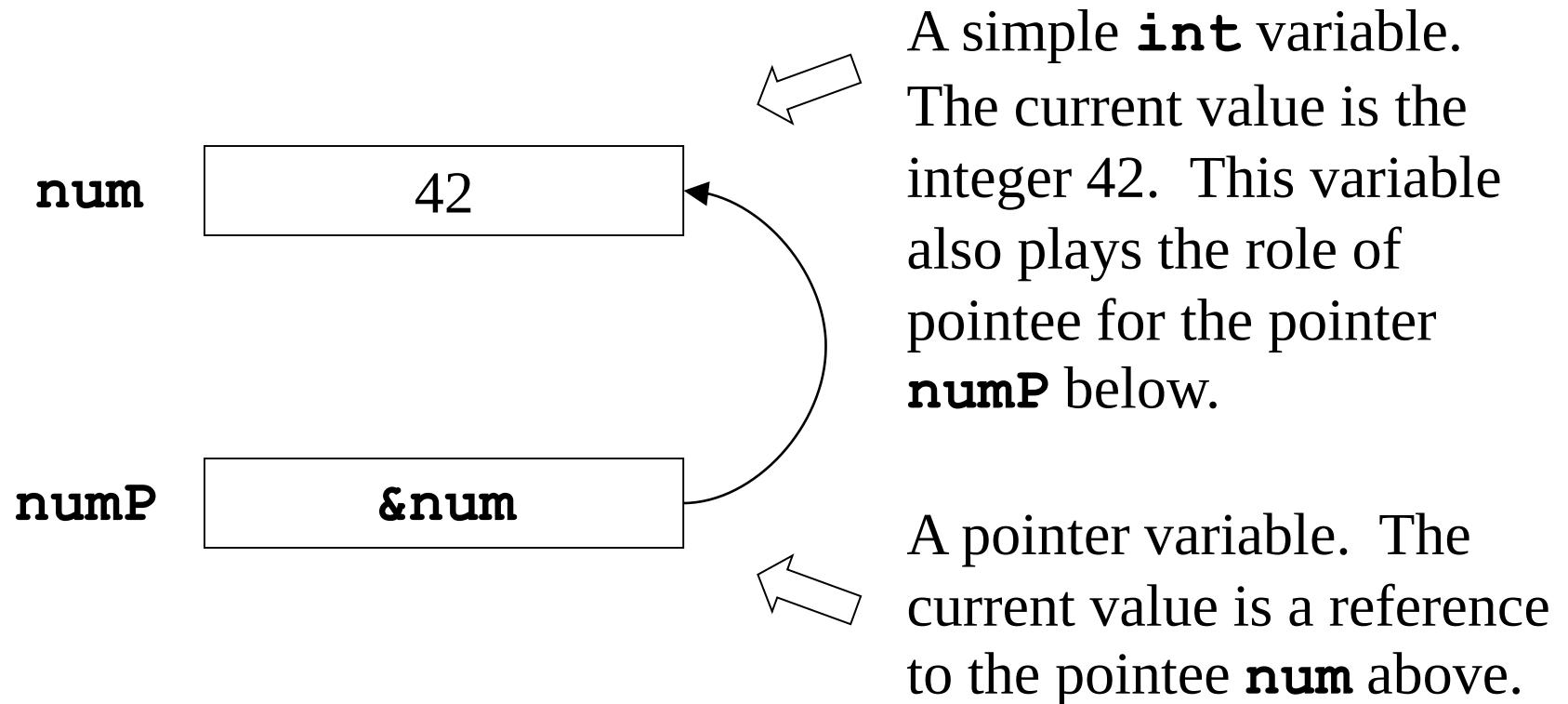
```
numP = &num;  
sumP = &sum;  
symP = &mySym;
```

```
*numP = 42;  
*sumP = 17.0;  
(*symP).val = 2;
```

```
printf("%i %3.1f %lu\n", num, sum, mySym.val) ;  
printf("%i %3.1f %lu\n", *numP, *sumP, (*symP).val) ;
```

```
-----  
42 17.0 2  
42 17.0 2
```

Example from Pointers & Memory



What form does data take?

- Integers

- ▶ -1, 0, 255, 65535, ...

- Floating point

- ▶ 1.5, 3.14159, 1E75, ...

- character strings

- ▶ "abc", "def"

- and that was about it in the old days

But real data is more complex

■ Airplane definition

- ▶ engine count, crew count, passenger capacity, range, operating cost per seat mile, ...

■ Student record

- ▶ name, student id, major, school address, home address, credits to date, current enrollment, ...

■ Major fields of study

- ▶ responsible department, curriculum, students, ...

Why have structs?

- Because the logical objects that you use in your programs are more complex than just a single int or double value
- A structured block lets you manipulate related data as one element

```
struct Symbol {  
    char *name;  
    unsigned long val;  
};  
  
struct Symbol oneSym;  
struct Symbol twoSym;  
struct Symbol mySym;  
  
oneSym.name = "one";  
oneSym.val = 1;  
  
twoSym.name = "two";  
twoSym.val = oneSym.val+1;
```

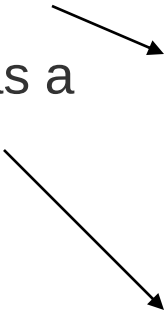
What is a *struct* variable?

- A single variable declared as a **struct** refers to a particular block of memory
- the individual fields are at fixed offsets from the start of the block

oneSym	char *
	unsigned long
twoSym	char *
	unsigned long
mySym	char *
	unsigned long

What can you do with a struct?

- The legal operations on a structure are
 - ▶ accessing its members
 - ▶ copying it or assigning to it as a unit
 - ▶ taking its address with &



```
struct Symbol oneSym;  
struct Symbol twoSym;  
struct Symbol mySym;  
  
oneSym.name = "one";  
oneSym.val = 1;  
  
twoSym.name = "two";  
twoSym.val = oneSym.val+1;  
  
mySym = twoSym;
```

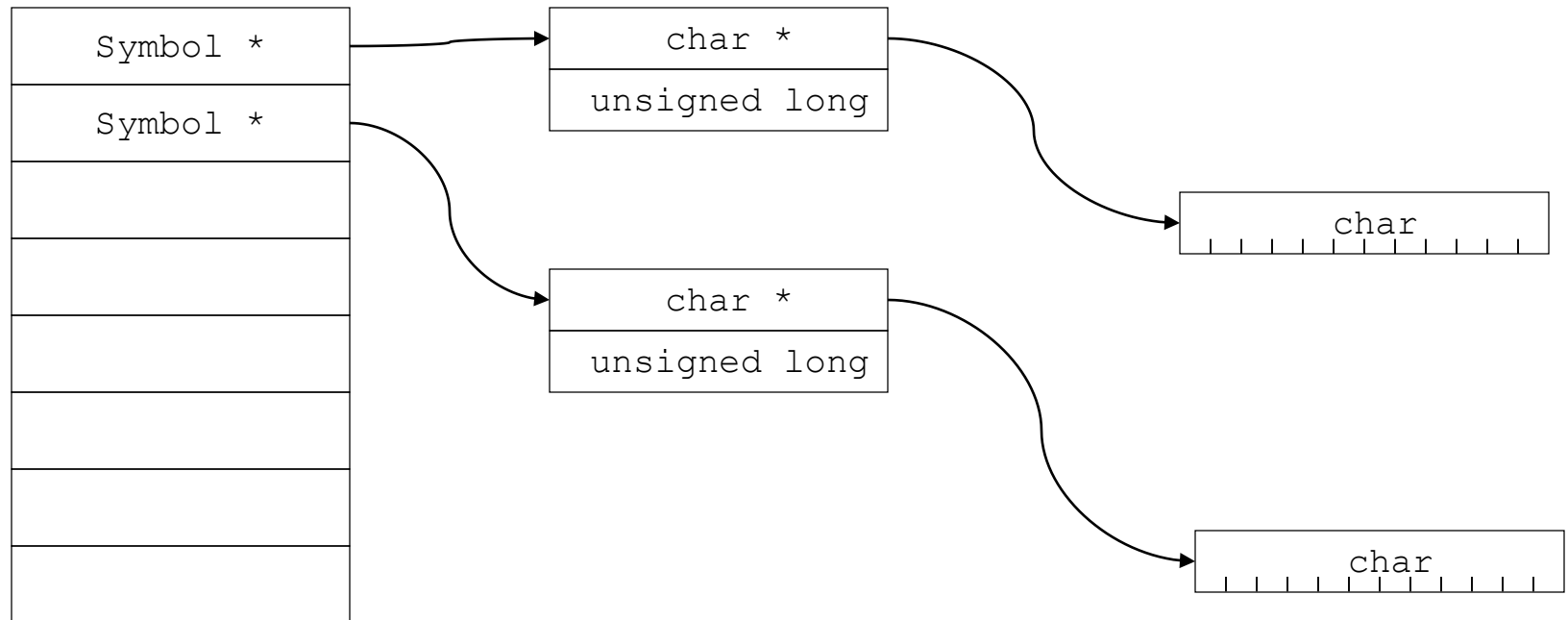
*How can **structs** be costly?*

- Copying a struct is a nice automatic feature
 - ▶ but it can lead to a lot of copying
- Our Symbol structs only require a few bytes
 - ▶ but imagine the size of some of the other examples - airplanes, student records, department descriptions
- Copying complete **structs** can get very costly very quickly

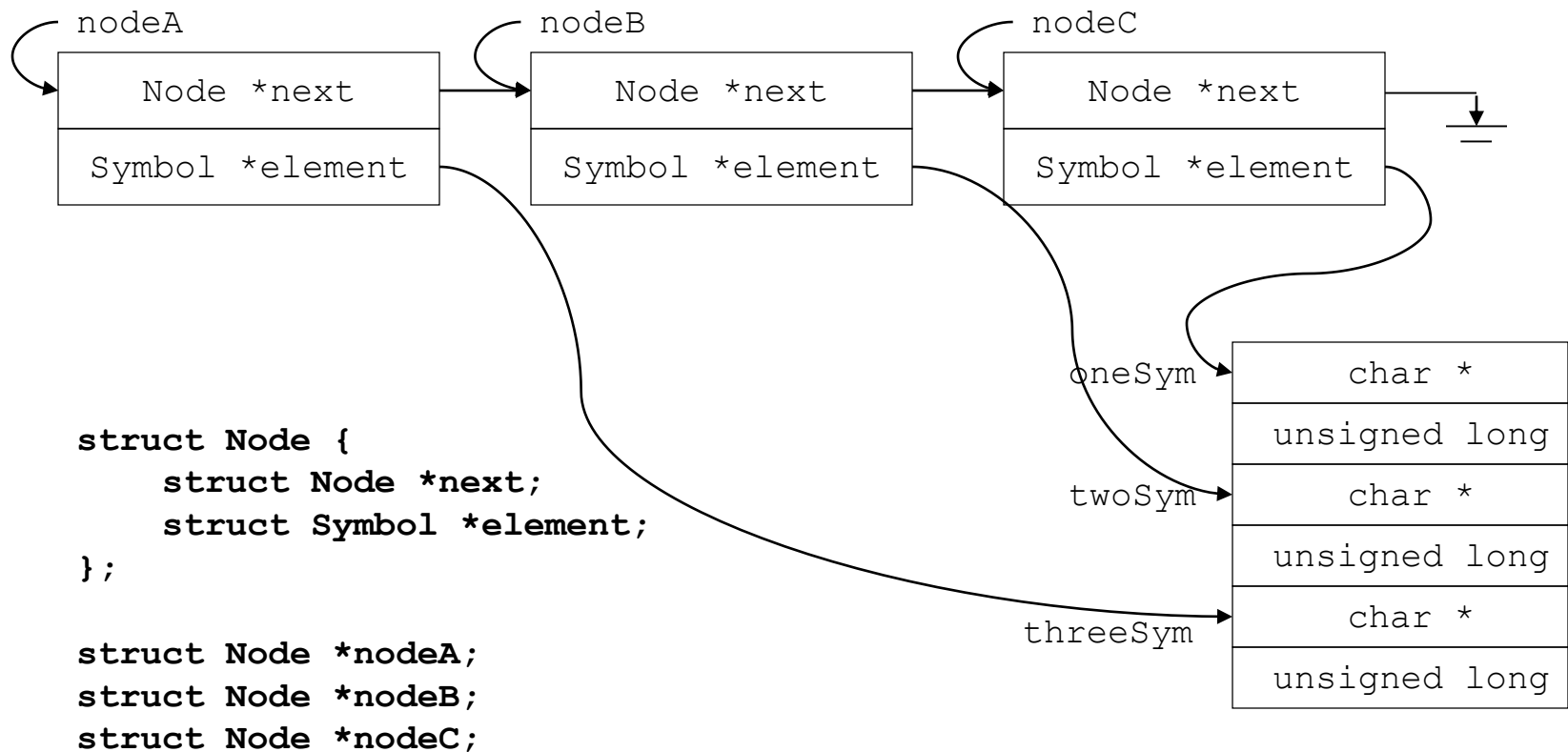
Pointers to the rescue

- Take the address of a struct variable and store it in a pointer variable
- Then you can manipulate the pointers, leaving the original data where it is and just moving pointer values around
- An array of pointer values is one way to define a list of objects (struct variables)

A short array of Symbol pointers



Pointers allow you to link objects



```
nodeA = malloc(sizeof(struct Node));  
FatalErrorMemory(nodeA);
```

```
nodeB = malloc(sizeof(struct Node));  
FatalErrorMemory(nodeB);
```

```
nodeC = malloc(sizeof(struct Node));  
FatalErrorMemory(nodeC);
```

```
(*nodeA).next = nodeB;  
(*nodeA).element = &oneSym;
```

```
nodeB->next = nodeC;  
nodeB->element = &twoSym;
```

```
nodeC->next = NULL;  
nodeC->element = &threeSym;
```

```
nodeP = nodeA;  
while (nodeP != NULL) {  
    printf("%p (%s) points to %p\n",  
        nodeP, nodeP->element->name, nodeP->next);  
    nodeP = nodeP->next;  
}
```

Three Linked Nodes

0x80498f0 (one) points to 0x8049900
0x8049900 (two) points to 0x8049910
0x8049910 (three) points to (nil)